

Issue 228 Test Logs And Defects

This file collates the full content of the main log, subagent logs, and defect files for PDF export.

Collated File: issue-228-test-log.md

Issue 228 Test Log

```
##
2026-
06-
25T00:20:10+01:00
##
2026-
06-
25T00:22:00+01:00
-
What
you
think
you
want
to
do
and
why
```


2026-
06-
25T00:20:10+01:00

Run
Loop
3 as
a
con-
sol-
da-
tion
and
gap-
review
loop.
Re-
cent
new
in-
for-
ma-
tion
is
shift-
ing
from
new
prod-
uct
de-
fects
into
cov-
er-
age
rec-
on-
cili-
a-
tion,
du-
pli-
cate
han-
dling,
and
de-
liv-
er-
able
com-
ple-
tion,
so

2026-
06-
25T00:20:10+01:00

the
ac-
tions
you
take
- in-
clude
urls,
steps
and
data
that
you
use,
de-
scribe
any
UI
in-
ter-
ac-
tions
that
you
make

2026-
06-
25T00:20:10+01:00

-
Gen-
er-
ated
at
least
10
Loop
3
ideas
in
the
main
re-
port.

-
Clas-
si-
fied
each
idea
as
execute-now
or
defer.

-
Exe-
cuted
the
cov-
er-
age
rec-
on-
cili-
a-
tion
items
by
re-
view-
ing
ex-
ist-
ing
logs,
de-
fect
files,
screen-
shots,

2026-
06-
25T00:20:10+01:00

the
ob-
ser-
va-
tions
and
re-
sults
that
you
make

2026-
06-
25T00:20:10+01:00

-
Loop
3
pro-
duced
no
new
must-
execute
prod-
uct
be-
hav-
ior
be-
yond
con-
soli-
da-
tion
and
arti-
fact
readi-
ness
checks.
- De-
ferred
ideas
are
mainly
ex-
haus-
tive
or
spe-
cial-
ist
ex-
ten-
sions:
ev-
ery
docs
code
block,
ev-
ery
do-
main
com-

2026-06-25T00:24:00+01:00

- What you think you want to do and why

Perform the mandatory final review loop before generating PDFs, checking story requirements, PR changed surface, accumulated logs, coverage model, sampled command families, docs reviewed, examples tried, defects found, and remaining gaps.

the actions you take - include urls, steps and data that you use, describe any UI interactions that you make

- Generated at least 10 final review ideas and classified them in the main report.
- Rechecked issue #228 scope against command definitions/help/docs/runtime consistency.
- Rechecked PR #243 changed surface against coverage from generator, app, docs, command coverage, validation, UX, responsive/accessibility, and enum canonicalization.
- Rechecked defect files and subagent logs in docs/testing/20260625/issue-228-001/.
- Prepared to generate test-logs-and-defects.md, issue-228-test-report.pdf, and the collated PDF after this final review.

the observations and results that you make

- Broad coverage is demonstrated across domain command families, faker/helper commands, newly changed enum commands, validators, structured parameters, docs with multiple examples, generator/app workflows, method picker, params editor, responsive behavior, and accessibility.
- Multiple loops completed, including final review.
- Recent loops are yielding mostly consolidation and deferred exhaustive ideas rather than genuinely new product behavior.
- Stopping is justified after artifact generation because required breadth, delegation, loop execution, final review, and reporting deliverables are satisfied.

2026-
06-
25T00:18:30+01:00

2026-
06-
25T00:17:30+01:00

2026-
06-
25T00:18:30+01:00

-
What
you
think
you
want
to
do
and
why

2026-
06-
25T00:18:30+01:00

Reconcile
a
con-
flict
be-
tween
my
early
text-
mode
enum
ob-
ser-
va-
tion
and
the
docs/enum
sub-
agent
re-
sults.
The
re-
port
should
not
carry
a
false
con-
firmed
de-
fect
if
clean-
state
evi-
dence
con-
tra-
dicts
it.

2026-
06-
25T00:18:30+01:00

the
ac-
tions
you
take
- in-
clude
urls,
steps
and
data
that
you
use,
de-
scribe
any
UI
in-
ter-
ac-
tions
that
you
make

```
##
2026-
06-
25T00:18:30+01:00
-
Opened
a
clean
Play-
wright
ses-
sion
at
https://eviltester.github.io/grid-table-editor/generator.html
-
Used
Edit
as
Text.
- Entered:
text
Status
enum("Open","Closed")
```

2026-
06-
25T00:18:30+01:00

-
Clicked
Preview.

-
Read
the
Output
Preview
value.

-
Deleted
the
ear-
lier
de-
fect
file
defects/issue-228-generator-text-mode-enum-does-not-preview.n

-
Up-
dated
the
re-
port
to
move
this
from
con-
firmed
de-
fects
to
an
in-
vali-
dated/suspicious
setup
note.

2026-
06-
25T00:18:30+01:00

the
ob-
ser-
va-
tions
and
re-
sults
that
you
make

-
Clean-
state
out-
put
gen-
er-
ated
cor-
rectly:
text
"Status"
"Open"
"Closed"
"Closed"
"Closed"
"Open"
"Open"
"Open"
"Open"
"Closed"
"Closed"

2026-
06-
25T00:18:30+01:00

-
This
matches
the
docs-
consistency
and
enum-
canonicalization
sub-
agent
ob-
ser-
va-
tions
that
enum
text-
mode
forms
work
on
the
cur-
rent
de-
ployed
com-
mit.

-
The
ear-
lier
text-
mode
enum
fail-
ure
was
likely
caused
by
in-
valid
setup,
stale
state,
or a
non-
supported
one-
i

2026-06-25T00:16:05+01:00

- What you think you want to do and why

Integrate command-coverage subagent results into the main session so the report reflects broad sampled command-family execution, not just the first main-agent enum defect pattern.

the actions you take - include urls, steps and data that you use, describe any UI interactions that you make

- Read `command-coverage-test-log.md`.
- Read command coverage defect files:
 - `defects/DEFECT-command-docs-location-cardinaldirection-abbreviated.md`
 - `defects/DEFECT-faker-doc-examples-do-not-execute.md`
- Compared command-coverage observations against the main-agent text-mode enum finding.

the observations and results that you make

- Command coverage successfully sampled docs/runtime behavior across enum, datatype boolean, finance, commerce, date, number, science, system, autoIncrement, helpers.fake, compact enum lists, and the domain picker.
- Command coverage confirms text-schema mode works for many documented two-line schemas and compact inline enum lists, so the main-agent text-mode issue is better framed as a repeatable enum command/form problem, not a blanket text-mode failure.
- Two additional confirmed docs/runtime mismatch defects were added by the subagent: `invalid location.cardinalDirection(abbreviated=true)` overview example and non-executable Faker Based Data examples.

2026-
06-
25T00:14:20+01:00

2026-
06-
25T00:12:45+01:00

2026-
06-
25T00:14:20+01:00

-
What
you
think
you
want
to
do
and
why

2026-
06-
25T00:14:20+01:00

Begin
Loop
1
broad
cov-
er-
age
on
the
de-
ployed
gen-
era-
tor
command-
entry
sur-
faces,
start-
ing
with
enum
be-
cause
PR
#243
heav-
ily
changes
enum
canon-
ical-
iza-
tion,
aliases,
val-
ida-
tors,
pars-
ing,
and
gen-
era-
tion.
17.

2026-
06-
25T00:14:20+01:00

the
ac-
tions
you
take
- in-
clude
urls,
steps
and
data
that
you
use,
de-
scribe
any
UI
in-
ter-
ac-
tions
that
you
make

##

2026-

06-

25T00:14:20+01:00

-

Opened

<https://eviltester.github.io/grid-table-editor/generator.html>

- In

row

mode,

en-

tered

col-

umn

status,

se-

lected

enum,

value

New, In

Progress, Done,

set

pre-

view

count

to 5,

and

clicked

Preview.

-

Clicked

Edit

as

Text

and

ob-

served

seri-

al-

ized

text

status

enum(New, "In

Progress", Done).

-

Tried

text-

mode

schema

status

enum(New, Done)\nflag

datatype.boolean\nrole

Admin, User, Guest

and

clicked

2026-
06-
25T00:14:20+01:00

the
ob-
ser-
va-
tions
and
re-
sults
that
you
make

2026-
06-
25T00:14:20+01:00

-
Row-
mode
enum
gen-
er-
ated
val-
ues
from
the
sup-
plied
enum
set;
ob-
served
out-
put
in-
cluded
New
and
In
Progress.

-
Row-
mode
to
text-
mode
seri-
al-
iza-
tion
pro-
duced
status
enum(New,"In
Progress",Done).

-
Text-
mode
~~enum~~
vari-
ants
failed
to
pro-
duce
pre-

2026-06-25T00:08:15+01:00

- What you think you want to do and why

Prove browser access and interaction before substantive testing, as required. Capture the deployed index metadata and any browser-console observations so the retest has clear evidence that it targeted the changed deployment.

the actions you take - include urls, steps and data that you use, describe any UI interactions that you make

- Opened `https://eviltester.github.io/grid-table-editor/` with Playwright CLI session `issue228-main`.
- Captured screenshot `screenshots/browser-proof-index.png`.
- Read Playwright console log `.playwright-cli/console-2026-06-24T23-08-06-624Z.log`.

the observations and results that you make

- Browser reached the deployed test environment successfully.
- Screenshot shows branch `codex/228-improve-command-definition`, commit `a3b39ddcfe0f`, built `2026-06-24T23:03:43.621Z`.
- One console error was observed: missing `https://eviltester.github.io/favicon.ico` with HTTP 404. This appears unrelated to issue `#228` command-definition behavior and is not currently treated as a defect for this review.

2026-
06-
25T00:05:15+01:00
-
What
you
think
you
want
to
do
and
why

Start

a

new

full

ex-

ploratory

retest

be-

cause

the

de-

ployed

test

en-

vi-

ron-

ment

meta-

data

changed

from

the

prior

mon-

i-

tored

base-

line.

The

pre-

vi-

ous

com-

pleted

re-

view

base-

line

was

../../20260624/issue-228-002/,

which

recorded

branch

codex/228-improve-command-definition,

com-

mit

fb9e8e2049e1,

built

2026-06-24T20:13:50.037Z.

The

live

en-

vi-

ron-

ment

the
ac-
tions
you
take
- in-
clude
urls,
steps
and
data
that
you
use,
de-
scribe
any
UI
in-
ter-
ac-
tions
that
you
make

-
Reread
C:\Users\mr_ri\.codex\attachments\9d4bd73f-06c0-410c-9a4e-68
-
Checked
de-
ploy-
ment
meta-
data
via
<https://eviltester.github.io/grid-table-editor/?monitor=2026>
-
Opened
the
de-
ployed
en-
vi-
ron-
ment
with
Play-
wright
CLI
ses-
sion
issue228-monitor
us-
ing
goto
<https://eviltester.github.io/grid-table-editor/>.
-
Cre-
ated
ses-
sion
fold-
ers
un-
der
docs/testing/20260625/issue-228-001/.
- Re-
viewed
GitHub
is-
sue
#2528
and
PR
#243
file
list
to
iden-

the
ob-
ser-
va-
tions
and
re-
sults
that
you
make

—
-
HTTP
meta-
data
re-
turned
200.
- De-
ployed
branch:
codex/228-improve-command-definition.
- De-
ployed
com-
mit:
a3b39ddcfe0f.
- De-
ployed
built
times-
tamp:
2026-06-24T23:03:43.621Z.
-
Play-
wright
reached
the
test
en-
vi-
ron-
ment
page
ti-
tled
Grid
Table
Editor
Test
Environment.
-
GitHub
is-
sue
#228
says
the
in²⁷
tent
is to
merge
com-
mand
defi-
ni-

Collated File: command-coverage-test-log.md

Command Coverage Test Log

2026-06-25T00:40:00+01:00 - Charter and Setup

Charter: deployed-environment exploratory QA retest for issue #228 / PR #243, focused on command coverage and example execution. Scope was broad positive sampling across domain command families, faker/helper commands, newly added/changed enum commands, commands with validators, structured/constrained parameters, default examples, parameterized examples, and commands with multiple docs/help examples.

Operating constraints followed: used only <https://eviltester.github.io/grid-table-editor/> and public deployed docs/pages. Did not run local verify, build, package-manager, or repo test commands. Did not edit app code.

Environment under test:

- URL: <https://eviltester.github.io/grid-table-editor/>
- Branch shown by deployed landing page: `codex/228-improve-command-definition`
- Commit shown by deployed landing page: `a3b39ddcfe0f`
- Build shown by deployed landing page: `2026-06-24T23:03:43.621Z`
- Browser automation: Playwright driving installed Chrome against deployed pages only.

Techniques and heuristics:

- Docs-as-oracle comparison: copy documented examples from public Docusaurus pages and execute through deployed generator preview.
- Risk-based sampling: prioritize commands touched by command-help/domain metadata refactor and enum work.
- Equivalence partitioning: cover defaults, named parameters, constrained numeric ranges, booleans, strings, enums, date windows, nested domain aliases, faker helpers, and auto-increment state.
- Positive broad sweep first, then isolate suspicious failures to avoid one bad command blocking unrelated coverage.
- Picker/help coverage: inspect live row-editor domain command select options for availability and absence/presence of changed commands.

2026-06-25T00:43:00+01:00 - Deployed App and Docs Discovery

Steps:

- Opened <https://eviltester.github.io/grid-table-editor/>.

- Opened `https://eviltester.github.io/grid-table-editor/generator.html`.
- Opened public docs pages under `https://eviltester.github.io/grid-table-editor/site/docs/`.
- Confirmed schema text format from `site/docs/test-data/Schema-Definition`.
- Confirmed domain catalog from `site/docs/test-data/domain/domain-test-data`.
- Read focused public docs pages for `datatype`, `person`, `location`, `finance`, `date`, `number`, `science`, `system`, `internet`, `commerce`, `autoIncrement`, and `faker-test-data`.

Observations:

- Generator page loaded as `Data Generator - AnyWayData`.
- Text-schema mode accepted two-line field definitions and compact inline `Name: values` definitions.
- Preview button generated visible CSV output in the `Output Preview` text area.
- Domain docs state that each domain page lists methods, arguments, and executable examples.

2026-06-25T00:48:00+01:00 - Smoke Schema Execution

Schema:

```
Company
company.name
Regex Generated Field
[A-Z]{3,6}[0-9]{0,6}
```

Steps:

- Switched generator to `Edit as Text`.
- Entered the schema above.
- Clicked `Generate Data`, then used `Preview`.

Result:

- `Generate Data` reached `Download ready: generated-data.csv`.
- `Preview` produced CSV with company names and regex-matching values such as `GOMS49`, `GTGX875335`, and `WITH00922`.
- No validation errors observed.

2026-06-25T00:52:00+01:00 - Broad Positive Batch Sampling

Batch A - docs smoke, enums, literal, regex, compact inline enum values:

```
Status
enum("Open","In Progress","Closed")
Build
literal(1.0.0)
Ticket Id
```

[A-Z]{3}-\d{4}

Browser: Chrome,Firefox,Safari

Theme: Light,Dark

Result:

- Passed.
- Output included `Status`, `Build`, `Ticket Id`, `Browser`, and `Theme`.
- Generated values stayed within expected enums, literal stayed 1.0.0, ticket IDs matched the expected pattern, and compact inline `Browser/Theme` enum values generated correctly.
- `Generate Combinations` became available when enum-like fields were present.

Batch B - changed datatype enum/boolean commands:

BoolDefault

`datatype.boolean()`

BoolProb

`datatype.boolean(probability=0.5)`

ApiStatus

`datatype.enum(values="active,inactive,pending")`

HttpMethod

`datatype.enum(values="GET,POST,PUT,PATCH")`

AliasEnum

`enum("High","Medium","Low")`

Result:

- Passed.
- Output included booleans as `true/false`.
- `datatype.enum` generated only configured values.
- `enum(...)` alias continued to generate only configured values.

Batch C - person/internet/location defaults and params:

Bio

`person.bio`

FirstFemale

`person.firstName(sex="female")`

FullName

`person.fullName`

EmailAda

`internet.email(firstName="Ada")`

DisplayName

`internet.displayName`

Lang2

`location.language.alpha2`

DirectionAbbrev

`location.cardinalDirection(abbreviated=true)`

Result:

- Failed because of `location.cardinalDirection(abbreviated=true)`.
- Error: Row 7: invalid domain params - Invalid keyword arguments: unknown named argument "abbreviated".
- Other commands in the batch were not judged from this combined run because the invalid command blocked preview.
- Follow-up isolation is recorded below.

Batch D - finance/commerce structured validators:

```
IbanGB
finance.iban(formatted=true, countryCode="GB")
IbanDE
finance.iban(formatted=false, countryCode="DE")
Account5
finance.accountNumber(length=5)
AmountMax
finance.amount(max=100)
Isbn10
commerce.isbn(variant=10)
PriceBound
commerce.price(dec=2, max=10, min=1, symbol="$")
```

Result:

- Passed.
- `IbanGB` generated formatted GB IBAN strings with spaces.
- `IbanDE` generated unformatted DE IBAN strings.
- `Account5` generated five-digit strings.
- `AmountMax` stayed below 100.
- `Isbn10` generated ISBN-10-style values.
- `PriceBound` generated \$-prefixed values between 1 and 10.

Batch E - date/number constrained params:

```
DateBetween
date.between(from=1577836800000, to=1609372800000)
BirthAge
date.birthdate(refDate=1577836800000, max=69, min=16, mode="age")
IntRange
number.int(min=32, max=47)
BinaryRange
number.binary(max=10, min=1)
FloatFixed
number.float(min=1, max=5, fractionDigits=2)
```

Result:

- Passed.

- Dates stayed in plausible configured windows.
- `number.int` stayed within 32..47.
- `number.float` produced two fractional digits and stayed within 1..5.
- `number.binary` produced binary strings with values corresponding to the configured numeric range.

Batch F - science/system nested domain leaf commands:

```
AtomicNum
science.chemicalElement.atomicNumber
ElementName
science.chemicalElement.name
UnitName
science.unit.name
UnitSymbol
science.unit.symbol
FileTxt
system.commonFileName(extension="txt")
CronYear
system.cron(includeYear=true)
```

Result:

- Passed.
- Output included atomic numbers, element names, unit names, unit symbols, `.txt` file names, and cron expressions.
- This covered newly surfaced nested domain leaf commands including `science.unit.name` and structured system parameters.

Batch G - faker/helper commands:

```
FakeSentence
helpers.fake("Hi, my name is {{person.firstName}} {{person.lastName}}!")
Mustache
helpers.mustache("I found {{count}} instances.", { count: () => `${this.number.int()}` })
FakerDirection
faker.location.cardinalDirection({ abbreviated: true })
```

Result:

- Failed as a combined batch.
- Errors:
- Row 2: invalid faker params - Invalid Faker API Call Unsafe faker rule syntax detected: requires complex argument parsing
- Row 3: invalid faker params - Invalid keyword arguments: too many positional arguments. Expected at most 0, received 1
- Follow-up isolation showed `helpers.fake(...)` works, while the published `helpers.mustache(...)` and `faker.location.cardinalDirection({ abbreviated: true })` examples fail.

Batch H - autoIncrement corrected examples:

Seq

```
autoIncrement.sequence(start=10, step=2)
```

SeqPad

```
autoIncrement.sequence(start=1, step=5, prefix="filename", suffix=".txt", zeropadding=3)
```

CreatedAt

```
autoIncrement.timestamp(start="2026-06-12T12:39:23Z", step=1, type="seconds")
```

Result:

- Passed.
- Seq generated 10, 12, 14,
- SeqPad generated filename001.txt, filename006.txt, filename011.txt, etc.
- CreatedAt generated second-by-second timestamps from 2026-06-12T12:39:23Z.

2026-06-25T01:03:00+01:00 - Suspicious Failure Isolation

Isolated `location.cardinalDirection(abbreviated=true)`:

Direction

```
location.cardinalDirection(abbreviated=true)
```

Result:

- Failed.
- Error: Row 1: invalid domain params - Invalid keyword arguments: unknown named argument "abbreviated".
- Public domain overview quick examples include Direction with `location.cardinalDirection(abbreviated=true)`.
- Public location domain page says `location.cardinalDirection` has no parameters.

Control using `location.direction(abbreviated=true)`:

Direction

```
location.direction(abbreviated=true)
```

Result:

- Passed.
- Output contained abbreviated directions such as SW, N, W, NW, SE, and E.
- This suggests the overview quick example likely meant `location.direction(abbreviated=true)`.

Isolated `helpers.fake(...)`:

Sentence

```
helpers.fake("Hi, my name is {{person.firstName}} {{person.lastName}}!")
```

Result:

- Passed.

- Output contained sentences such as Hi, my name is Marisa McLaughlin!.

Isolated `helpers.mustache(...)`:

Sentence

```
helpers.mustache("I found {{count}} instances.", { count: () => `${this.number.int()}` })
```

Result:

- Failed.
- Error: Row 1: invalid faker params - Invalid Faker API Call
Unsafe faker rule syntax detected: requires complex argument parsing.
- This exact example appears on the public Faker Based Data docs page.

Isolated direct faker object-style example:

Direction

```
faker.location.cardinalDirection({ abbreviated: true })
```

Result:

- Failed.
- Error: Row 1: invalid faker params - Invalid keyword arguments: too many positional arguments. Expected at most 0, received 1.
- This exact example appears on the public Faker Based Data docs page.

2026-06-25T01:10:00+01:00 - Picker and Help Coverage

Steps:

- Returned to grid schema mode.
- Changed the first row data type to `domain`.
- Inspected the live domain command picker.
- Saved screenshot: docs/testing/20260625/issue-228-001/screenshots/command-coverage-domain-

Observed picker data:

- Domain picker has 253 options.
- Sample options include `airline.aircraftType`, `animal.*`, `autoIncrement.sequence`, `autoIncrement.timestamp`, `book.*`, `commerce.*`, `company.*`, `datatype.boolean`, `datatype.enum`, `date.*`, `finance.*`, `internet.*`, `location.*`, `number.*`, `person.*`, `science.*`, `string.*`, and `system.*`.
- Changed/high-risk commands visible in picker included `autoIncrement.sequence`, `autoIncrement.timestamp`, `datatype.boolean`, `datatype.enum`, `finance.iban`, `internet.httpMethod`, `location.cardinalDirection`, and `location.direction`.
- `helpers.*` was not present in the domain picker, consistent with domain docs saying `helpers.*` is intentionally faker-only.

Coverage:

- Domain picker availability: covered.
- Domain/faker split: covered at picker and runtime level.
- Multiple examples/help: covered via public docs pages and isolated execution.

Findings

Confirmed defects:

- **DEFECT-command-docs-location-cardinaldirection-abbreviated.md:** public domain overview quick example uses `location.cardinalDirection(abbreviated=true)`, but deployed runtime rejects `abbreviated`; `location.direction(abbreviated=true)` works and the location domain detail page documents `location.cardinalDirection` as no-parameter.
- **DEFECT-faker-doc-examples-do-not-execute.md:** two public Faker Based Data direct examples fail in deployed generator: `helpers.mustache(...)` with callback object and `faker.location.cardinalDirection({ abbreviated: true })`.

Positive findings:

- Core schema text parsing works for two-line and compact inline forms.
- `enum(...)`, raw compact enum lists, `datatype.enum(values="...")`, and `datatype.boolean(...)` all generated valid sampled outputs.
- Structured/named validator examples for finance, commerce, date, number, system, and autoIncrement executed successfully.
- Nested command examples for science and system executed successfully.
- `helpers.fake(...)` executed successfully as a faker helper command.
- Domain picker contains broad command coverage and includes changed commands.

Deferred Items

- Did not exhaust all 253 domain picker options.
- Did not execute every example from every domain page; sampled representative defaults and parameterized examples across high-risk families.
- Did not do negative validation beyond failures encountered naturally from published examples; that belongs to the negative-validation lane.
- Did not cross-check Storybook method picker stories; this lane focused on deployed generator and public docs/runtime execution.
- Did not inspect app code or local generated metadata by design.

New Ideas

- Add a deployed-docs smoke harness that extracts public docs code blocks and runs them through the generator preview, tagging expected executable

examples versus documentation-only snippets.

- Add a docs lint rule that flags object-literal faker examples if the deployed parser only supports named-argument syntax.
- Add a domain overview consistency check so quick examples are validated against each command page metadata.
- Add picker inventory snapshot comparison for PRs that change command metadata, with allowed/hidden command assertions such as `helpers.*` absent from domain picker.
- Add a lightweight example matrix for `datatype.enum` forms: raw compact list, `enum(...)`, `datatype.enum(values="...")`, quoted multi-word values, and HTTP method enum values.
- Add a visual separator/accessibility review of the preview table because concatenated visible table text is hard to read in automation output, though CSV output is correct.

Collated File: negative-validation-test-log.md

Negative Validation Test Log

2026-06-25T00:11:29+01:00

Charter

Subagent retest for issue #228 / PR #243 on the deployed environment only:

- Target: <https://eviltester.github.io/grid-table-editor/>
- Branch observed on test environment: `codex/228-improve-command-definition`
- Commit observed on test environment: `a3b39ddcfe0f`
- Build observed on test environment: `2026-06-24T23:03:43.621Z`
- Focus: negative validation and malformed parameter testing.
- In scope: invalid enum syntax, empty enum values, malformed named values params, bad quotes, stray commas, unknown commands, disallowed helper commands, invalid numeric/probability/date constraints, and usefulness/consistency of validation and error messages in generator/app flows.
- Out of scope: local verify/build/package-manager/repo test commands and app code edits.

Techniques And Heuristics

- Deployed-environment exploratory testing with browser automation.
- Positive-anchor then mutation testing: first confirmed a documented valid `datatype.enum(values="active,inactive,pending")` schema, then mutated one dimension at a time.

- Equivalence partitioning around empty, missing, malformed, unknown, and boundary/out-of-range parameter values.
- Boundary analysis for `probability`, `multipleOf`, and date `days`.
- Cross-surface consistency sampling between standalone `generator.html` Preview and main `app.html` Test Data Generate.
- Error-message usefulness review: checked whether invalid input failed early with actionable validation, silently failed, generated corrupted data, or emitted generic `**ERROR**`.
- Recheck loop: repeated likely defects from a fresh generator tab before filing.

Browser Proof And Setup

1. Opened `https://eviltester.github.io/grid-table-editor/`.
2. Confirmed build metadata showed branch `codex/228-improve-command-definition`, commit `a3b39ddcfe0f`, built `2026-06-24T23:03:43.621Z`.
3. Clicked `Open generator.html`.
4. Confirmed the generator page loaded with schema editor, `Generate Data`, and `Preview` controls.
5. Captured initial generator screenshot:
 - `screenshots/negative-validation-generator-initial.png`
6. Used published docs pages as public references:
 - `https://eviltester.github.io/grid-table-editor/site/docs/test-data/domain/datatype`
 - `https://eviltester.github.io/grid-table-editor/site/docs/test-data/domain/date`
 - `https://eviltester.github.io/grid-table-editor/site/docs/test-data/domain/number`
7. A guessed docs URL, `https://eviltester.github.io/grid-table-editor/site/docs/test-data/don` returned 404; treated as a bad lookup, not an app finding.

Generator Baseline

Test data:

```
status
datatype.enum(values="active,inactive,pending")
```

Steps:

1. Opened `generator.html`.
2. Switched to `Edit as Text`.
3. Entered the schema above.
4. Used `Preview` with a small row count.

Observation:

- Preview generated only expected enum values: `active`, `inactive`, and `pending`.
- No validation errors appeared.
- This was used as the positive anchor for invalid mutations.

Generator Negative Matrix

Raw observed matrix output was saved to:

- `negative-validation-matrix-results.json`

Clean recheck output for likely defects was saved to:

- `negative-validation-recheck-results.json`

Test data and observations:

Area	Test data	Generator observation
Empty enum values	<code>datatype.enum(values="")</code>	Preview produced no output and no visible validation message on two runs.
Missing enum values arg	<code>datatype.enum()</code>	Preview produced no output and no visible validation message on two runs.
Stray middle enum comma	<code>datatype.enum(values="active, pending")</code>	Active, pending with Row 1: invalid enum value - Invalid keyword arguments: bare values are not allowed; wrap strings in quotes. Failure was visible, but message did not specifically explain empty enum values or stray comma.
Leading enum comma	<code>datatype.enum(values=", active, pending")</code>	Active, pending as stray comma.
Trailing enum comma	<code>datatype.enum(values="active, pending,")</code>	Active, pending as stray comma.
Bad enum quote	<code>datatype.enum(values="active, pending")</code>	Active, pending successfully, including corrupted value <code>\\\"active</code> , with no validation error. Rechecked from a clean tab.

Area	Test data	Generator observation
Unknown enum named arg	<code>datatype.enum(valuez="active,pending")</code>	Successfully rather than rejecting unknown <code>valuez</code> . Initial run produced <code>valuez=active</code> and <code>pending</code> ; clean recheck produced <code>pending</code> values only. No validation error appeared.
Unquoted named values	<code>datatype.enum(values=active,pending)</code>	Given <code>active</code> and <code>pending</code> ; treated as permissive behavior rather than a defect because public docs mention unquoted <code>enum active,inactive,pending</code> authoring.
Raw comma empty middle	<code>active,,pending</code>	Treated as literal/regex-like value and generated <code>active,,pending</code> ; not filed because text schema type inference may intentionally treat this as non-enum without explicit enum syntax.
Unknown command	<code>datatype.noSuchCommand()</code>	Generated regex-like values such as <code>datatypeYnoSuchCommand</code> ; not filed from text-mode alone because unknown untyped rule appears to fall through to regex inference. Should be retested in an explicitly domain-typed row-mode path later.

Area	Test data	Generator observation
Disallowed helper attempt	<code>helpers.arrayElement(values, "fake, pending")</code>	Correctly failed, pending visible message: Row 1: invalid faker params - Invalid Faker API Call Unsafe faker rule syntax detected: requires complex argument parsing.
Probability > 1	<code>datatype.boolean(probability=2)</code>	Generated true values and no validation error on two runs, despite public docs saying probability is between 0 and 1.
Probability < 0	<code>datatype.boolean(probability=-1)</code>	Generated false values and no validation error on two runs, despite public docs saying probability is between 0 and 1.
Probability text	<code>datatype.boolean(probability="abc")</code>	Correctly failed visibly as invalid domain params, though the message focused on bare values rather than numeric probability.
Number min > max	<code>number.int(min=10,max=10)</code>	Correctly failed with actionable message: argument "min" must be less than or equal to argument "max".
Number multipleOf zero	<code>number.int(min=1,max=10,multipleOf=0)</code>	Correctly failed containing **ERROR** with no validation message on two runs.
Number max text	<code>number.int(max=abc)</code>	Correctly failed visibly as invalid domain params, though the message focused on bare values rather than numeric max.

Area	Test data	Generator observation
Date negative days	<code>date.recent(days=-7)</code>	Generated rows containing **ERROR** with no validation message on two runs.
Date refDate text	<code>date.soon(refDate=notadate)</code>	Correctly failed visibly as invalid domain params, though the message focused on bare values rather than refDate.

Main App Sample

Raw app sample output was saved to:

- `negative-validation-app-sample-results.json`

Steps:

1. Opened <https://eviltester.github.io/grid-table-editor/app.html?negative-app-sample=1>.
2. Expanded Test Data.
3. Switched schema editor to Edit as Text.
4. Ran a small targeted sample through the app Generate flow.

App observations:

Test data	App observation
<code>datatype.enum(values="active,inactive")</code>	Baseline passed; grid updated with four expected enum rows.
<code>datatype.enum(values="")</code>	App showed Generate failed. Check console for details. and left previous grid data visible. This is a visible failure, but the message is not actionable.
<code>datatype.enum(values="active,pending")</code>	App showed Generate complete. Grid updated. and generated corrupted <code>\\\"active</code> values.
<code>datatype.boolean(probability=2)</code>	App showed Generate complete. Grid updated. and generated all true.
<code>number.int(min=1,max=10,multipleOf=1)</code>	App showed Generate complete. Grid updated. and populated the grid with **ERROR** rows.

Screenshot captured:

- `screenshots/negative-validation-app-multipleof-zero-error-success.png`

Confirmed Findings

Confirmed defect file:

- `defects/negative-validation-defects.md`

Summary:

- Malformed enum quote is accepted in both generator Preview and app Generate, producing corrupted values while reporting success/no error.
- `datatype.boolean(probability=2)` and `datatype.boolean(probability=-0.1)` are accepted despite the documented 0 to 1 range.
- `number.int(multipleOf=0)` produces ****ERROR**** output while app reports successful generation.
- `date.recent(days=-7)` produces ****ERROR**** output in generator Preview instead of a validation message.
- Empty/missing enum values are inconsistent: generator Preview silently produces no output/no visible error, while app Generate fails with only **Check console for details**.
- Unknown enum named `arg valuez` is accepted rather than rejected, producing output with no validation error.

Validation/Error Message Quality Notes

- Strong messages observed:
 - `argument "min" must be less than or equal to argument "max"` is specific and actionable.
 - The disallowed helper command produced a clear unsafe/complex parsing error.
- Weak or inconsistent messages observed:
 - Stray comma enum cases fail, but the message says bare values are not allowed; it does not directly mention empty enum values or stray commas.
 - Non-numeric bare values fail, but messages focus on quoting rather than the expected numeric parameter type.
 - App failures that say **Check console for details** are less useful than row-level schema validation messages.
 - ****ERROR**** rows are not useful as user-facing validation feedback and can be mistaken for generated data.

Coverage

Covered:

- Standalone generator text schema mode.
- Main app Test Data text schema mode.

- Public datatype/date/number docs pages for parameter names and examples.
- Valid enum baseline.
- Enum empty/missing/stray-comma/leading-comma/trailing-comma/bad-quote/unknown-named-arg/unquoted-named-list/raw-comma-list.
- Unknown command in untyped text schema mode.
- Disallowed helper command attempt.
- Boolean probability above range, below range, and non-numeric.
- Number `min > max`, `multipleOf=0`, and non-numeric max.
- Date negative days and non-numeric `refDate`.

Not covered:

- Full row-mode parameter editor modal for every invalid parameter.
- Explicitly domain-typed row-mode unknown command, because text-mode inference may intentionally treat unknown untyped rules as regex.
- Storybook surfaces.
- Mobile/responsive behavior for validation messages.
- Console stack trace analysis; app message asked users to check console, but this charter focused on visible validation usefulness.

Deferred Items

- Retest unknown commands in row mode with explicit `domain` and `faker` type selections so regex fallback cannot mask unknown command validation.
- Exercise the parameter editor modal directly for enum `values`, boolean `probability`, number `multipleOf`, and date `days`.
- Check whether **Generate Data** file creation has the same failures as Preview, because this pass prioritized Preview and app grid generation.
- Add accessibility-focused checks for whether validation messages receive focus or are announced after failed generation.
- Compare docs examples with in-app help examples for the same invalid/boundary params.

New Ideas

- Add a small negative examples section to docs for enum values, probability ranges, and numeric constraints.
- Normalize empty enum error wording across empty values, missing values arg, leading comma, middle comma, and trailing comma.
- Treat ****ERROR**** as a failed generation state and block success messages until the underlying row-level cause is surfaced.
- Validate `multipleOf > 0` before calling the generator.
- Validate `date.recent(days)` and `date.soon(days)` as non-negative integers before calling `faker`.
- Validate boolean `probability` range inclusively as `0 <= probability <= 1`.

- Reject unknown named args for commands with documented parameter lists.
- Distinguish “bare value needs quotes” from “expected number” and “unknown parameter” errors.
- Preserve previous grid/output only with a clear “generation failed; previous data retained” message.
- Add shared validation tests that assert generator Preview and app Generate report the same error for the same schema.
- Add a compact UI affordance near schema text mode showing row number and offending rule when validation fails.
- Consider replacing `Check console for details` with a visible expandable detail panel in the app.

Collated File: docs-consistency-test-log.md

Docs/Help/Content Consistency Retest Log

2026-06-25T00:55:00+01:00

Charter

Docs/help/content consistency across app and published docs for issue #228 / PR #243. Compare the deployed public docs, in-app/generator help, picker examples, and practical runtime behavior for changed command families. Focus areas: `datatype.enum` aliases, named `values` params, raw comma examples, hidden/disallowed faker helpers, stale `generate.html` vs `generator.html` links, and examples that no longer match runtime.

Environment And Constraints

- Deployed environment only: <https://eviltester.github.io/grid-table-editor/>.
- Deployment metadata observed from the existing issue log and landing page: branch `codex/228-improve-command-definition`, commit `a3b39ddcfe0f`, built 2026-06-24T23:03:43.621Z.
- Did not run local `verify/build/package-manager/repo` test commands.
- Browser automation: attempted Playwright MCP first, but the MCP bridge failed to attach with `Invalid URL: undefined`; switched to Chrome DevTools Protocol against the deployed site.
- HTTP checks used only public deployed pages/assets.

Techniques And Heuristics

- Link consistency checks between public docs and deployed routes.

- Documentation example testing against current runtime.
- Cross-surface oracle comparison: public docs text, in-app/generator help text, command picker options, params dialog behavior, schema text serialization, and generated preview output.
- Alias equivalence partitioning for enum spellings.
- Negative/permission-oriented checks for helper-only faker commands in domain vs faker contexts.
- Retest against prior artifact folder to avoid duplicating stale findings.

Pages And Assets Reviewed

- <https://eviltester.github.io/grid-table-editor/>
- <https://eviltester.github.io/grid-table-editor/app.html>
- <https://eviltester.github.io/grid-table-editor/generator.html>
- <https://eviltester.github.io/grid-table-editor/generate.html>
- <https://eviltester.github.io/grid-table-editor/site/>
- <https://eviltester.github.io/grid-table-editor/site/app.html>
- <https://eviltester.github.io/grid-table-editor/site/generator.html>
- <https://eviltester.github.io/grid-table-editor/site/generate.html>
- <https://eviltester.github.io/grid-table-editor/site/docs/intro>
- <https://eviltester.github.io/grid-table-editor/site/docs/test-data/test-data-generation>
- <https://eviltester.github.io/grid-table-editor/site/docs/test-data/generate-to-file>
- <https://eviltester.github.io/grid-table-editor/site/docs/test-data/Schema-Definition>
- <https://eviltester.github.io/grid-table-editor/site/docs/test-data/domain/domain-test-d>
- <https://eviltester.github.io/grid-table-editor/site/docs/test-data/domain/datatype/>
- <https://eviltester.github.io/grid-table-editor/site/docs/test-data/faker-test-data>
- Deployed JS assets sampled: `generator-8qkDmNIq.js`, `generator-script-qDlNeETi.js`, `domain-command-provider-DnyhZrzJ.js`, `schema-conversion-CGLqX1_q.js`, `app-CThAVrql.js`, `script-DIr1JVbM.js`.

Examples Tried

- `status / datatype.enum(values="active,inactive,pending")` in generator text mode.
- `status / enum("active","inactive","pending").`
- `status / enum active,inactive,pending.`
- `status / active,inactive,pending.`
- `status / "active","inactive","pending".`
- `status / awd.datatype.enum("active","inactive","pending").`
- `status / datatype.enum(active,inactive,pending).`
- Row-mode domain command picker: selected `datatype.enum`, opened params dialog, entered `active,inactive,pending`, applied params.
- Domain picker filtering: searched domain options for `datatype.enum`, `helpers.*`, `faker.helpers.*`, and `arrayElement`.
- Faker picker filtering: searched faker options for helper entries.
- Published faker docs examples:

- `helpers.fake("Hi, my name is {{person.firstName}} {{person.lastName}}!")`
- `faker.helpers.fake("Hi, my name is {{person.firstName}} {{person.lastName}}!")`
- `helpers.mustache("I found {{count}} instances.", { count: () =>${this.number.int()})`
- Domain misuse check: `domain.helpers.fake("Hi")`.

Consistency Observations

- `generator.html` works at both root and nested site paths; `generate.html` returns 404 at both root and nested site paths.
- Public datatype docs are live at `/site/docs/test-data/domain/datatype/`.
- Public datatype docs describe `datatype.enum(values="..."), enum("..."), enum value1,value2`, raw comma lists, quoted raw comma lists, and `awd.datatype.enum(...)` compatibility.
- Runtime generated values successfully for all sampled enum forms above. The raw comma run sampled only two of three values in 10 preview rows, but that is compatible with random sampling rather than a defect.
- The row-mode domain picker includes `datatype.enum`.
- The `datatype.enum` params dialog:
 - Opens with command label `awd.domain.datatype.enum`.
 - Shows `values` as a required checkbox in the Req column.
 - Keeps Apply disabled until a value is entered.
 - Applies entered values as `enum(active,inactive,pending)` in the public schema text, matching the intended public/internal normalization split.
- Domain command picker hides faker helper-only commands: no `helpers.arrayElement`, `faker.helpers.*`, or other `helpers.*` entries appeared in domain options.
- Faker picker intentionally exposes helper commands, including `helpers.fake`, `helpers.mustache`, and `helpers.arrayElement`.
- Domain docs explicitly say `helpers.*` is faker-only and `domain.helpers.fake(...)` is invalid.
- Runtime rejects `domain.helpers.fake("Hi")` with visible guidance: `helpers.*` is faker-only; use `faker.helpers.*` plus an unknown domain command message.
- `helpers.fake(...)` and `faker.helpers.fake(...)` examples from faker docs generate expected names in runtime.
- The previously filed `issue-228-generator-text-mode-enum-does-not-preview.md` was not reproduced for the enum text-mode cases sampled in this pass on the same deployed commit.

Defects Filed

- `defects/docs-still-reference-generate.html.md`
- `defects/faker-docs-helpers-mustache-example-rejected.md`

Defects

Medium: Published docs still describe `generate.html`, but deployed route is `generator.html` Confirmed on public docs pages:

- `/site/docs/test-data/test-data-generation`
- `/site/docs/test-data/generate-to-file`
- `/site/docs/test-data/Schema-Definition`

Observed examples include visible headings/text such as `Generate to File` (`generate.html`), page metadata saying `Use generate.html`, and schema-definition text saying the schema editor is in `app.html` and `generate.html`. Root and nested `generate.html` routes return 404, while `generator.html` routes return 200.

Medium: Published `helpers.mustache` faker example is rejected by deployed runtime The public faker docs show:

```
helpers.mustache("I found {{count}} instances.", { count: () => `${this.number.int()}` })
```

When entered in deployed generator text mode, preview output stays empty and the visible error says the faker params are invalid because unsafe faker rule syntax requires complex argument parsing. This makes the published example unusable as written.

Deferred Items

- I did not exhaustively click every docs sidebar entry or every helper command. This pass focused on the changed command families called out in the charter.
- I did not verify screenshots because CDP text/DOM/runtime evidence was sufficient for these content consistency defects.
- I did not retest app embedded generation deeply beyond public app shell/link/bundle checks because the charter emphasis was docs/help/content consistency and generator command examples.
- I did not edit the pre-existing text-mode enum defect even though the sampled enum text-mode cases now work; leaving that for the parent retest owner to reconcile against their exact steps/screenshot.

New Ideas

- Add an automated deployed-docs link check that fails on `generate.html` references unless explicitly grandfathered.
- Add a docs-example smoke runner that extracts code blocks from `datatype` and `faker-test-data` docs and previews/generates one row in the deployed generator.
- Split faker helper docs into safe UI-supported examples vs full Faker API examples that require complex JavaScript callbacks.

- In the faker picker, mark helpers that are exposed but likely constrained by safe parser support, or link to a docs section explaining the supported argument subset.
- Add a docs note near `helpers.mustache` if callback/object-literal arguments are intentionally unsupported in browser schema text.
- Add a small in-app help text check to ensure help links point to public docs routes that return 200.
- Preserve an explicit docs term map: public page name `Generate` to `File`, route `generator.html`, schema mode names, and app labels.
- Add a regression test for domain picker filtering that verifies `datatype.enum` appears while `helpers.*` does not.
- Add a regression test for faker picker filtering that verifies intended helper commands appear only in faker context.
- Reconcile the pre-existing text-mode enum defect against the current behavior; if fixed, mark as resolved in the parent report.

Collated File: ux-regression-test-log.md

UX Regression Retest Log - Issue #228 / PR #243

Charter

Deployed-environment exploratory UX/usability and workflow regression retest for issue #228 / PR #243.

Target only: <https://eviltester.github.io/grid-table-editor/>

Do not run local `verify/build/package-manager/repo` test commands. No app code edits.

Focus areas:

- `generator.html`
- `app.html` embedded test-data/schema workflow
- Method picker search/filter
- Params editor modal
- Help panels
- Import/export schema workflow
- Focus/keyboard paths where relevant
- Whether recent validation/accessibility fixes introduce friction

Techniques And Heuristics

- Deployed-browser exploratory testing using a real Chrome browser controlled through Playwright.
- Clean-page retries for defect confirmation where practical.
- Workflow-first UX heuristics: discoverability, recovery, clear feedback, minimal interruption.
- Accessibility heuristics: focus entry/exit, keyboard operation, icon-only accessible names, modal focus return.
- Regression heuristics: compare equivalent generator/app embedded controls, exercise fixed validation/accessibility surfaces rather than only happy-path generation.
- Evidence rule: only file defects for behavior observed on the deployed target and supported by screenshots or DOM state.

Environment And Browser Control Proof

- Date: 2026-06-25.
- Browser: local Chrome executable driven headlessly.
- Deployed target opened at <https://eviltester.github.io/grid-table-editor/>.
- Proof interaction: clicked `Open generator.html` from the deployed index and reached <https://eviltester.github.io/grid-table-editor/generator.html>.
- Evidence:
- `screenshots/proof-home.png`
- `screenshots/proof-generator-click.png`

Sequential Steps And Observations

1. Opened deployed index page.
2. Confirmed index title: `Grid Table Editor Test Environment`.
3. Clicked `Open generator.html`.
4. Confirmed generator title: `Data Generator - AnyWayData`.
5. Scouted generator page controls: schema editor, text mode, schema file actions, generation controls, options, preview, help icons, method picker trigger after choosing `faker`.
6. Generated enum data in `generator.html` using `Status`, type `enum`, values `Open,Closed`, preview count 5.
7. Observed output preview with header and five generated values. No console errors were observed in that flow.
8. Switched schema type to `faker`.
9. Opened method picker using `Select faker command`.
10. Observed method picker modal with search field, category chips, method tiles, Cancel, and Apply.
11. Searched `email` while `Faker` chip was active.
12. Observed zero visible method tiles and no empty-state message. This is a UX friction observation, not filed as a defect because switching to `All`

- correctly finds email-related methods.
13. Switched method picker to **All**, searched **email**.
 14. Observed **internet.email** and **internet.exampleEmail**.
 15. Applied **internet.email**.
 16. Observed selected command returned to the schema row and picker closed.
 17. Reopened method picker, switched to **All**, searched **number.int**, selected and applied it.
 18. Opened params editor for **number.int**.
 19. Confirmed initial focus moved into the params table at **min value**.
 20. Entered **min=1, max=3**, applied params.
 21. Generated preview data and observed values between 1 and 3 using params (**min=1,max=3**).
 22. Opened method picker via keyboard from **Select faker command**.
 23. Confirmed focus initially moved to the picker search field and early Tab stops remained inside the picker modal.
 24. Pressed Escape to close method picker.
 25. Confirmed focus dropped to **<body>** instead of returning to the command-picker trigger. Filed defect.
 26. Opened params editor via keyboard from params button.
 27. Confirmed focus initially moved to **min value**.
 28. Pressed Escape to close params editor.
 29. Confirmed focus dropped to **<body>** instead of returning to the params trigger. Same defect scope.
 30. Exercised generator help icons.
 31. Observed help buttons toggled **aria-expanded** from **false** to **true** and help/disclosure content remained visible without blocking the workflow.
 32. Exported generator schema with one enum field using **Save Schema File**.
 33. Observed download suggested filename **schema.txt**.
 34. Saved artifact locally as **schema-export-schema.txt** in this QA folder.
 35. Loaded the exported schema into a fresh **generator.html** page using the deployed file input.
 36. Confirmed schema row restored as **Status, enum, Open,Closed** with no visible errors.
 37. Opened **app.html**.
 38. Scouted app page: grid toolbar, collapsed **Test Data** section, collapsed **Import / Export** section, text preview tabs/options.
 39. Expanded **Test Data**.
 40. Filled embedded schema row with **Status, enum, Open,Closed**.
 41. Set visible generate count to **3**.
 42. Clicked the embedded generate button using its deployed **data-role**.
 43. Confirmed grid updated to **Total rows: 3**, header **Status**, and three generated enum rows.
 44. Observed several compact app icon buttons have no text, **aria-label**, or **title**, despite being enabled user-facing controls. Filed defect.
 45. Clicked **Set Text From Grid**.
 46. Confirmed CSV text preview was populated from the grid.

47. Cleared the table, edited preview text to "Status"\n"Imported", and clicked **Set Grid From Text**.
48. Confirmed grid imported text back as one row with header **Status** and value **Imported**.

Confirmed Findings

Medium - Modal Escape Close Does Not Restore Focus To Trigger

Defect file: `defects/issue-228-modal-focus-not-restored.md`

Observed in both method picker and params editor. Keyboard users lose their place after closing the modal with Escape because focus lands on `<body>` rather than the control that opened the modal.

Evidence:

- `screenshots/method-picker-keyboard-after-esc.png`
- `screenshots/params-editor-keyboard-after-esc.png`

Medium - App Embedded Icon-Only Controls Lack Accessible Names

Defect file: `defects/issue-228-app-icon-controls-missing-accessible-names.md`

Several enabled app-page icon-only controls in the embedded instructions/schema/import-export workflows have empty visible text and no `aria-label/title`. This creates friction for screen reader users and makes compact controls hard to understand without visual icon interpretation.

Evidence:

- `screenshots/app-initial.png`
- `screenshots/app-generate-new-table.png`

Coverage

- Browser-control proof from deployed index to `generator.html`.
- Generator enum schema happy path.
- Generator faker method picker open/search/filter/apply.
- Method picker category interaction with **Faker** and **All**.
- Method picker no-result state observed.
- Params editor open/fill/apply for `number.int`.
- Generator output preview after params application.
- Keyboard open/close for method picker.
- Keyboard open/close for params editor.
- Generator help icon/disclosure behavior.
- Generator schema save/load file workflow.
- App **Test Data** embedded schema generation workflow.
- App grid-to-text and text-to-grid workflow.
- App compact icon-button accessible-name inspection.

Deferred Ideas

- Test method picker full Tab cycle through the end of the modal to confirm wrap behavior, not only early tab stops.
 - Test mobile/narrow viewport usability for method picker and params editor, especially category chip overflow.
 - Compare accessible names with a browser accessibility snapshot if a dedicated ally tool is available.
 - Test app embedded schema Save/Load file buttons functionally once accessible names are fixed or surfaced.
 - Recheck whether method picker no-result searches should display an explicit “No methods found” message.
-

Collated File: responsive-accessibility-test-log.md

Responsive And Accessibility Test Log

2026-06-25 00:13 Europe/London - Responsive/mobile and accessibility retest

Charter

Retest the deployed environment for issue #228 / PR #243 with a responsive/mobile and accessibility focus. Stay deployed-only at <https://eviltester.github.io/grid-table-editor/>. Do not run local verify, build, package-manager, or repo test commands. Focus on index, generator, app, method picker, params/editor modal areas, help/disclosure controls, keyboard tab order, labels/names, focus restoration, and accessibility regressions tied to recent fixes. Create defect markdown only for confirmed defects.

Techniques And Heuristics

- Deployed-environment exploratory testing against the public GitHub Pages URL.
- Responsive viewport sampling across mobile portrait, mobile landscape, tablet, and desktop.
- Keyboard-only tab-order tracing from clean page states.
- Chrome accessibility-tree inspection for focusable control names.
- DOM heuristics for overflow, focusable controls, dialogs, details/summary disclosures, labels, and likely unlabeled controls.
- Screenshot-backed visual review for layout and focus-risk areas.
- Risk-based prioritization around PR #243 accessibility, picker, and schema UI changes.

Viewports And Test Data

- Mobile portrait: 360 x 740, mobile emulation, device scale factor 2.
- Mobile landscape: 740 x 360, mobile emulation, device scale factor 2.
- Tablet: 768 x 1024, mobile emulation, device scale factor 2.
- Desktop: 1280 x 900, desktop emulation, device scale factor 1.
- Primary routes sampled:
 - <https://eviltester.github.io/grid-table-editor/>
 - <https://eviltester.github.io/grid-table-editor/site/>
 - <https://eviltester.github.io/grid-table-editor/app.html>
 - <https://eviltester.github.io/grid-table-editor/generator.html>
 - <https://eviltester.github.io/grid-table-editor/combinatorial.html>
 - <https://eviltester.github.io/grid-table-editor/webmcp.html>
 - <https://eviltester.github.io/grid-table-editor/writer-schema.html>
 - <https://eviltester.github.io/grid-table-editor/storybook/index.html>

Browser Control Proof

- Chrome DevTools MCP and Playwright MCP were blocked by existing browser/profile conflicts, so I used Chrome's DevTools Protocol directly with a temporary profile.
- Browser interaction proof captured by navigating the deployed index and clicking a visible route/control.
- Evidence:
 - [screenshots/responsive-a11y-browser-proof-click.png](#)
 - [responsive-accessibility-cdp-results.json](#)

Observations

- The index/test-environment landing page remained responsive at 360 px, 740 px, 768 px, and 1280 px. Tab order covered the route links repeatedly without layout breakage.
- The Docusaurus `site/` page adapted to mobile navigation. The only overflow heuristic hit was the off-canvas skip link position, which is expected for that pattern and was not filed.
- Generator responsive layout stacked schema controls and generation options without visible horizontal scrolling at 360 px.
- App responsive layout kept the main grid and test-data/editor sections reachable in a narrow viewport, though the page is long and dense on mobile.
- Export/settings details appeared visually contained in screenshots, even where DOM heuristics reported offscreen hidden/absolute panel geometry.
- Storybook loaded and adapted to the sampled sizes; the offscreen search label is a common visually-hidden label pattern and was not filed.
- WebMCP mobile had a long method name/card near the right edge. It did not produce page-level horizontal scroll, so I left it as a watch item rather than a defect.

Confirmed Findings

- `defects/issue-228-generator-schema-row-tab-order-trap.md`
 - From the generator schema row **Column Name** input, **Tab** moves to **body**, then back through row action buttons and the same input. It does not advance to the field type picker, value input, constraints, or generation controls.
- `defects/issue-228-controls-missing-accessible-names.md`
 - Chrome’s accessibility tree exposes empty names for the generator schema type combobox, generator preview searchbox, and app grid searchbox.

Coverage

- Covered responsive behavior on index, site, app, generator, combinatorial, WebMCP, writer-schema, and Storybook routes.
- Covered mobile and desktop tab traces on the main sampled routes.
- Covered generator help/disclosure controls at a basic click level.
- Covered generator/app accessible-name risks using Chrome’s accessibility tree for confirmed controls.
- Covered layout overflow heuristics and screenshot evidence for sampled routes.
- Created 38 `responsive-a11y-*` screenshots plus a structured JSON capture.

Deferred Ideas

- Recheck modal focus restoration after opening a concrete params editor modal from a populated command that exposes editable parameters.
- Exercise the method picker with a mouse and keyboard after adding multiple field rows and changing field families.
- Run a screen-reader-oriented pass with a real accessibility extension or manual assistive-tech session.
- Check zoom at 200% and Windows high contrast mode.
- Add a targeted mobile pass for WebMCP method cards with very long method names.
- Recheck app import/export file controls and generated download controls for names after the parallel import/export lanes settle.

Files Written

- `responsive-accessibility-test-log.md`
- `responsive-accessibility-cdp-results.json`
- `live-responsive-a11y-cdp.mjs`
- `defects/issue-228-generator-schema-row-tab-order-trap.md`
- `defects/issue-228-controls-missing-accessible-names.md`
- Screenshots under `screenshots/responsive-a11y-*.png`

Collated File: enum-canonicalization-test-log.md

Enum Canonicalization Retest Log

Date: 2026-06-25 Environment: <https://eviltester.github.io/grid-table-editor/>
Charter: Retest the PR #243 changed enum surface for alias/canonicalization and schema round-trip behavior across `generator.html` and `app.html`, focusing on `datatype.enum`, enum aliases, raw comma values, `enum(...)` forms, named values parameters, generated output, help text, saved/exported schema representation, imported schema representation, and whether the UI normalizes or preserves values consistently.

Techniques And Heuristics

- Equivalence partitioning across enum spellings: raw comma values, `enum(...)`, `datatype.enum(...)`, and alias-style forms where discoverable.
- Round-trip testing: enter schema/value, sync or save/export, reload/import, and compare visible representation.
- Cross-surface consistency: compare generator and app behavior for the same enum specs.
- Oracle checks from visible output: verify generated rows use only declared enum values and inspect whether schema text remains public-facing.
- Error-seeking inputs: values containing commas, whitespace, mixed case, and named `values=` parameter forms.
- Documentation/help comparison: compare help text and UI labels with observed accepted syntax.

Test Matrix

ID	Surface	Input/Action	Expected Oracle	Status	Evidence/Notes
M1	generator.htm	Raw comma enum values	Generated values limited to declared set; saved/schema UI behavior noted	Covered	Generated constrained values; save normalized to <code>enum(...)</code> .

ID	Surface	Input/Action	Expected Oracle	Status	Evidence/Notes
M2	generator.html	<code>enum(...)</code> public form	Accepted and preserved in public schema text where applicable	Covered	Generated con- strained values; saved as public <code>enum(...)</code> .
M3	generator.html	<code>datatype.enum(...)</code> canonical form	Accepted for runtime generation; public UI does not leak confusing internal spelling unless in- tentionally docu- mented	Covered	Accepted for generation; save/switch normal- ized to public <code>enum(...)</code> .
M4	generator.html	Named <code>values=</code> parameter form	Either accepted consis- tently or rejected with clear help/error	Covered	<code>datatype.enum(values="...")</code> and <code>enum(values="...")</code> both generated declared values.
M5	app.html	Schema grid/text entry with enum forms	UI syncs without stale <code>domain/enum</code> mismatch and pre- serves/normalizes consis- tently	Covered	No stale mismatch or sync recursion observed.

ID	Surface	Input/Action	Expected Oracle	Status	Evidence/Notes
M6	app.html	Export/save then im- port/load schema	Round- trip representa- tion is stable and generated output remains con- strained	Covered	Imported raw comma and <code>datatype.enum(values="...");</code> saved as public <code>enum(...)</code> .
M7	Public help/docs	Enum syntax help	Help documents accepted public syntax and does not contradict UI behavior	Covered	In-app help and datatype docs align with sampled accepted syntax.
M8	Cross- surface	Same enum specs in generator and app	Same values generated from same logical enum spec	Covered	Raw comma, <code>enum(...)</code> , <code>datatype.enum(...)</code> , named <code>values=</code> , and <code>awd.datatype.enum(...)</code> behaved consis- tently.
M9	generator.htm	Documented aliases: <code>enum</code> <code>value1,value2</code> quoted lists, <code>awd.datatype.enum(...)</code> , <code>awd.domain.datatype.enum(...)</code>	Accepted or clearly rejected in the docs	Covered	Documented aliases generated declared values; sampled aliases nor- malized on save.

Sequential Log

- 2026-06-25 00:10 Europe/London: Started retest as deployed-environment-only subagent. Constraints confirmed: no local verify/build/package-manager/repo test commands; only deployed site and public pages/docs; artifact writes only under requested QA folder.
- 2026-06-25 00:12 Europe/London: Memory context reviewed. Important oracle: internal/runtime canonicalization may use `datatype.enum(...)`, while public schema text should round-trip as `enum(...)`; prior failure mode to watch for is **Expected: "enum" Received: "domain"** or schema sync recursion.
- 2026-06-25 00:14 Europe/London: Proved deployed browser access by opening <https://eviltester.github.io/grid-table-editor/generator.html>, clicking the page, reading title **Data Generator - AnyWayData**, and saving screenshot `screenshots/enum-proof-generator.png`.
- 2026-06-25 00:16 Europe/London: Scouted `generator.html`. Visible surfaces include schema grid/text toggle, schema file load/save, enum/literal/regex/domain/faker source type selector, schema constraints, managed stored schemas, generate data, generate combinations, settings, preview, output preview, and data-table preview.
- 2026-06-25 00:18 Europe/London: Ran generator text-mode preview cases: raw comma `open,closed,pending`, public `enum(open,closed,pending)`, internal `datatype.enum(open,closed,pending)`, `datatype.enum(values="open,closed,pending")`, and `enum(values="open,closed,pending")`.
- 2026-06-25 00:20 Europe/London: Generator output for all five base cases used only the declared `open`, `closed`, and `pending` values. No validation messages or stale `domain/enum` mismatch appeared.
- 2026-06-25 00:22 Europe/London: Ran generator representation cases by switching text input to schema/grid mode and saving schema. `enum(...)`, `datatype.enum(...)`, `datatype.enum(values="...")`, and raw comma forms all mapped to grid source type `enum`, grid value `open,closed,pending`, and saved as `Status\nenum(open,closed,pending)`.
- 2026-06-25 00:24 Europe/London: Proved `app.html` access by opening <https://eviltester.github.io/grid-table-editor/app.html>, clicking the page, reading title **Test Data Generator and Table Editor for Markdown, CSV, JSON, Gherkin and HTML - AnyWayData**, and saving screenshot `screenshots/enum-proof-app.png`.
- 2026-06-25 00:26 Europe/London: Scouted `app.html`. Relevant surfaces include collapsed Test Data details, generation rows/mode controls, schema grid/text toggle, schema file load/save, grid-to-enum schema generation, import/export preview, grid sync buttons, and data grid output.
- 2026-06-25 00:28 Europe/London: Initial app fill attempt correctly exposed that the schema textarea is hidden while Test Data details are collapsed. Opened `details[data-role="test-data-details"]` and then used the visible schema mode toggle for the real user path.
- 2026-06-25 00:31 Europe/London: Ran app generation and representa-

tion cases for public `enum(...)`, `datatype.enum(...)`, raw comma, and `datatype.enum(values="...")`. Each generated 10 grid rows using only declared values and mapped back to schema grid source type `enum` with value `open,closed,pending`.

- 2026-06-25 00:33 Europe/London: App schema save/switch behavior normalized `datatype.enum(...)`, raw comma, and named-values inputs to public `Status\nenum(open,closed,pending)`, matching generator behavior.
- 2026-06-25 00:35 Europe/London: Added import fixtures `import-datatype-enum.schema` and `import-raw-comma.schema` in the QA folder. Imported both into generator and app using the visible Load Schema File control.
- 2026-06-25 00:37 Europe/London: Imported schema observations: both generator and app initially showed the imported source text in the schema text area, parsed it into grid source type `enum` with value `open,closed,pending`, and saved back out as `Status\nenum(open,closed,pending)`.
- 2026-06-25 00:39 Europe/London: Help/docs pass. In-app enum help says comma-separated enum values, `enum active,inactive,pending`, params field values, and example `enum active,inactive,pending`. Public datatype docs mention `enum(value1,value2)`, `enum value1,value2`, `datatype.enum(value1,value2)`, raw comma lists, quoted lists, named values=, `awd.datatype.enum(...)`, and canonical `awd.domain.datatype.enum`.
- 2026-06-25 00:42 Europe/London: Alias generation pass in generator: `enum open,closed,pending`, `"open","closed","pending"`, `enum("open","closed","pending")`, `awd.datatype.enum(open,closed,pending)`, and `awd.domain.datatype.enum(open,closed,pending)` all generated only declared values with no messages.
- 2026-06-25 00:44 Europe/London: Alias representation pass: `enum open,closed,pending` and `awd.datatype.enum(open,closed,pending)` mapped to schema grid source type `enum`, value `open,closed,pending`, and saved/switch-normalized to `Status\nenum(open,closed,pending)` in generator. `awd.datatype.enum(...)` behaved the same in app.
- 2026-06-25 00:46 Europe/London: Quoted comma-in-value probe: `City\nenum("New York, NY","Paris")` generated only `New York, NY` and `Paris`; grid value became `"New York, NY",Paris`; saved schema became `City\nenum("New York, NY",Paris)`. This is acceptable based on observed generation and parser preservation of the value containing a comma, but worth broader follow-up for quote-style preservation expectations.

Observations

- Browser proof and screenshots are stored under `screenshots/`, including `enum-proof-generator.png`, `enum-proof-app.png`, generator/app case screenshots, import screenshots, and docs screenshots.
- Generator accepts raw comma enum values, public `enum(...)`, internal `datatype.enum(...)`, named values=, `enum value1,value2`, quoted

lists, and documented `awd.*.enum(...)` aliases.

- App accepts the sampled raw comma, public `enum(...)`, internal `datatype.enum(...)`, named `values=`, and `awd.datatype.enum(...)` forms when Test Data details are opened and schema text mode is used.
- Generated output stayed within declared enum values across generator preview and app grid generation. No observed output included command names, parameter wrappers, or split artifacts for the sampled normal cases.
- Public representation is stable after schema-grid sync/save: sampled aliases and internal forms normalize to `enum(open,closed,pending)` rather than leaking `datatype.enum(...)`.
- Imported schema behaves consistently with typed schema: imported raw comma and `datatype.enum(values="...")` parse into enum grid controls and save as public `enum(...)`.
- In-app help and public datatype docs are broadly aligned with runtime behavior for enum aliases and named `values=`.
- The guessed public docs URL `site/docs/test-data/schema` returned GitHub Pages 404, but this was a tester-discovered guessed URL, not a linked product URL in this pass.

Findings

- No confirmed enum canonicalization defects found in this gap charter.
- No defect markdown files were created because all observed behaviors were consistent with the current oracle or were deferred as follow-up risks rather than confirmed defects.

Coverage

- M1 `generator.html` raw comma values: Covered. Generated values were constrained and schema save normalized to `enum(...)`.
- M2 `generator.html` public `enum(...)`: Covered. Generated values were constrained and schema save preserved public `enum(...)`.
- M3 `generator.html` `datatype.enum(...)`: Covered. Accepted for generation and normalized to public `enum(...)` on schema save/switch.
- M4 `generator.html` named `values=`: Covered for `datatype.enum(values="...")` and `enum(values="...")`. Both generated declared values; `datatype.enum(values="...")` normalized to `enum(...)`.
- M5 `app.html` schema grid/text entry: Covered. No stale domain/enum mismatch, recursion, or validation message observed.
- M6 `app.html` export/import schema: Covered through schema file import/save for raw comma and `datatype.enum(values="...")`.
- M7 public help/docs: Covered for in-app enum help and public datatype docs. The docs support the accepted alias set.
- M8 cross-surface consistency: Covered for raw comma, `enum(...)`, `datatype.enum(...)`, named `values=`, and `awd.datatype.enum(...)`.
- M9 documented aliases: Covered on generator for `enum value1,value2`,

quoted lists, `awd.datatype.enum(...)`, and `awd.domain.datatype.enum(...)`; covered on app for `awd.datatype.enum(...)`.

Deferred Ideas

- Broaden quote-style preservation checks for enum values containing commas. Current probe generated correct values, but save normalized `enum("New York, NY", "Paris")` to `enum("New York, NY", "Paris")`.
- Test `awd.domain.datatype.enum(...)` in `app.html` as well as generator; generator passed, and app passed the shorter `awd.datatype.enum(...)` alias.
- Exercise Generate Combinations/n-wise with mixed enum aliases on both surfaces, especially two enum columns authored with different spellings.
- Repeat import/export with multi-column schemas mixing enum aliases, constraints, and non-enum domain/faker fields.
- Check whether stored schema history preserves the same public normalization as file save/load.

Collated Defect Files

Collated Defect File: DEFECT-command-docs-location-cardinaldirection-abbreviated.md

Defect: Domain overview documents invalid `location.cardinalDirection(abbreviated=true)` example

Summary

The public Domain Test Data overview shows a quick example using `location.cardinalDirection(abbreviated=true)`, but the deployed generator rejects `abbreviated` for `location.cardinalDirection`. The location domain detail page documents `location.cardinalDirection` as having no parameters, and the equivalent parameterized command that works is `location.direction(abbreviated=true)`.

Environment

- Deployed app: <https://eviltester.github.io/grid-table-editor/generator.html>
- Public docs: <https://eviltester.github.io/grid-table-editor/site/docs/test-data/domain/domain-overview.html>

- Public detail docs: <https://eviltester.github.io/grid-table-editor/site/docs/test-data/domain>
- Branch shown by deployed landing page: `codex/228-improve-command-definition`
- Commit shown by deployed landing page: `a3b39ddcfe0f`
- Build shown by deployed landing page: `2026-06-24T23:03:43.621Z`

Steps to Reproduce

1. Open <https://eviltester.github.io/grid-table-editor/site/docs/test-data/domain/domain-t>
2. Note the quick example:

Direction

```
location.cardinalDirection(abbreviated=true)
```

3. Open <https://eviltester.github.io/grid-table-editor/generator.html>.
4. Switch schema editor to Edit as Text.
5. Enter:

Direction

```
location.cardinalDirection(abbreviated=true)
```

6. Click Preview.

Actual Result

Preview fails with:

Row 1: invalid domain params - Invalid keyword arguments: unknown named argument "abbreviated"

Expected Result

Either the documented quick example should execute successfully, or the overview should use the valid command:

Direction

```
location.direction(abbreviated=true)
```

Control test:

Direction

```
location.direction(abbreviated=true)
```

This passed and generated abbreviated directions such as SW, N, W, NW, SE, and E.

Notes for Fix

The location detail page says `location.cardinalDirection` has no parameters and documents `location.direction(abbreviated=true)` as the parameterized abbreviated-direction example. The likely fix is to update the Domain Test Data overview quick example from `location.cardinalDirection(abbreviated=true)` to `location.direction(abbreviated=true)`.

Collated Defect File: DEFECT-faker-doc-examples-do-not-execute.md

Defect: Two public Faker Based Data examples do not execute in deployed generator

Summary

The public **Faker Based Data** docs page presents direct faker examples that appear intended to be executable schema examples. Two of those examples fail in the deployed generator:

- `helpers.mustache("I found {{count}} instances.", { count: () => ${this.number.int()}})`
- `faker.location.cardinalDirection({ abbreviated: true })`

The adjacent `helpers.fake(...)` example does execute successfully.

Environment

- Deployed app: <https://eviltester.github.io/grid-table-editor/generator.html>
- Public docs: <https://eviltester.github.io/grid-table-editor/site/docs/test-data/faker-test-examples.html>
- Branch shown by deployed landing page: `codex/228-improve-command-definition`
- Commit shown by deployed landing page: `a3b39ddcfe0f`
- Build shown by deployed landing page: `2026-06-24T23:03:43.621Z`

Steps to Reproduce: `helpers.mustache`

1. Open <https://eviltester.github.io/grid-table-editor/generator.html>.
2. Switch schema editor to Edit as Text.
3. Enter the documented example:

Sentence

```
helpers.mustache("I found {{count}} instances.", { count: () => `${this.number.int()}` })
```

4. Click Preview.

Actual Result: `helpers.mustache`

Preview fails with:

Row 1: invalid faker params - Invalid Faker API Call Unsafe faker rule syntax detected: requ

Steps to Reproduce: direct faker object parameter

1. Open <https://eviltester.github.io/grid-table-editor/generator.html>.
2. Switch schema editor to **Edit as Text**.
3. Enter the documented example:

Direction

```
faker.location.cardinalDirection({ abbreviated: true })
```

4. Click **Preview**.

Actual Result: direct faker object parameter

Preview fails with:

Row 1: invalid faker params - Invalid keyword arguments: too many positional arguments. Exp

Expected Result

Public docs examples that are presented as schema examples should execute successfully in the deployed generator, or the docs should be changed to supported schema syntax.

Control Example

The adjacent docs example below passed:

Sentence

```
helpers.fake("Hi, my name is {{person.firstName}} {{person.lastName}}!")
```

Observed output included rows such as:

Hi, my name is Marisa McLaughlin!

Notes for Fix

If object-literal/callback faker arguments are intentionally unsupported by the schema parser, the docs should avoid presenting them as executable schema examples. If direct faker examples should be executable, the parser/runtime needs support for these documented argument shapes or the docs should provide the equivalent supported named-argument/domain form.

Collated Defect File: docs-still-reference-generate-html.md

Published Docs Still Reference `generate.html`

Summary

Several published docs pages still describe the generator page as `generate.html`, but the deployed environment exposes `generator.html`. Both root and nested `generate.html` URLs return 404.

Environment

- Landing page: <https://eviltester.github.io/grid-table-editor/>
- Branch: `codex/228-improve-command-definition`
- Commit: `a3b39ddcfe0f`
- Build: 2026-06-24T23:03:43.621Z

Steps To Reproduce

1. Open <https://eviltester.github.io/grid-table-editor/site/docs/test-data/test-data-generation-to-file>
2. Search the page text for `generate.html`.
3. Open <https://eviltester.github.io/grid-table-editor/site/docs/test-data/generate-to-file>
4. Search the page text/metadata for `generate.html`.
5. Open <https://eviltester.github.io/grid-table-editor/site/docs/test-data/Schema-Definition>
6. Search the page text for `generate.html`.
7. Compare route availability:
 - <https://eviltester.github.io/grid-table-editor/generate.html>
 - <https://eviltester.github.io/grid-table-editor/generator.html>
 - <https://eviltester.github.io/grid-table-editor/site/generate.html>
 - <https://eviltester.github.io/grid-table-editor/site/generator.html>

Expected Result

Published docs should consistently refer to the currently deployed generator route, `generator.html`, or avoid page-file names if the route is no longer meant to be user-facing.

Actual Result

- `test-data/test-data-generation` contains visible text such as `Generate to File (generate.html)`.
- `test-data/generate-to-file` contains metadata/text such as `Use generate.html`.
- `test-data/Schema-Definition` says the schema editor is in `app.html` and `generate.html`.

- <https://eviltester.github.io/grid-table-editor/generate.html> returns 404.
- <https://eviltester.github.io/grid-table-editor/site/generate.html> returns 404.
- <https://eviltester.github.io/grid-table-editor/generator.html> returns 200.
- <https://eviltester.github.io/grid-table-editor/site/generator.html> returns 200.

Severity

Medium.

This is a docs/content consistency regression that can send users to a dead page or teach the wrong route name for the changed generator surface.

Notes

The app and generator shell links sampled in this pass point to `generator.html`; the stale `generate.html` references appear concentrated in the published docs content.

Collated Defect File: `faker-docs-helpers-mustache-example-rejected.md`

Published `helpers.mustache` Faker Example Is Rejected By Runtime

Summary

The published faker docs include a `helpers.mustache(...)` example with an object-literal callback argument, but the deployed generator rejects that exact example as `unsafe/too complex`.

Environment

- URL: <https://eviltester.github.io/grid-table-editor/generator.html>
- Docs page: <https://eviltester.github.io/grid-table-editor/site/docs/test-data/faker-test>
- Branch: `codex/228-improve-command-definition`
- Commit: `a3b39ddcfe0f`
- Build: `2026-06-24T23:03:43.621Z`

Steps To Reproduce

1. Open <https://eviltester.github.io/grid-table-editor/site/docs/test-data/faker-test-data>
2. Find the published example:

```
helpers.mustache("I found {{count}} instances.", { count: () => `${this.number.int()}`
```
3. Open <https://eviltester.github.io/grid-table-editor/generator.html>.
4. Click Edit as Text.
5. Enter this schema:

```
sentence
helpers.mustache("I found {{count}} instances.", { count: () => `${this.number.int()}`
```
6. Click Preview.

Expected Result

The docs example should generate preview values, or the docs should avoid presenting it as a supported schema example in the browser generator.

Actual Result

The preview output remains empty. The visible runtime error says:

Invalid Faker API Call Unsafe faker rule syntax detected: requires complex argument parsing

Control Checks

- `helpers.fake("Hi, my name is {{person.firstName}} {{person.lastName}}!")` generated names successfully.
- `faker.helpers.fake("Hi, my name is {{person.firstName}} {{person.lastName}}!")` generated names successfully.
- `domain.helpers.fake("Hi")` was rejected with visible guidance that `helpers.*` is faker-only, which matches the domain docs.

Severity

Medium.

The docs advertise an example that users can copy directly, but the current deployed generator rejects it. This is especially risky because issue #228 / PR #243 changed command definitions/help and users may rely on docs examples as the oracle for supported helper syntax.

Notes

If complex callback arguments are intentionally unsupported in browser schema text, the docs should use a safe `helpers.mustache` example or explicitly label this as a Faker API illustration rather than a supported generator schema rule.

Collated Defect File: issue-228-app-icon-controls-missing-accessible-names.md

App embedded icon-only controls lack accessible names

Severity

Medium

Area

`app.html` embedded instructions, test-data schema, stored-schema, and import/export controls.

Environment

Deployed target: <https://eviltester.github.io/grid-table-editor/app.html>

Date observed: 2026-06-25

Summary

Several enabled icon-only controls on `app.html` have no visible text, `aria-label`, or `title`. They appear as compact icon buttons visually, but their accessible name is empty. This makes the controls difficult or impossible to identify for screen reader users and increases keyboard-discovery friction.

Reproduction

1. Open <https://eviltester.github.io/grid-table-editor/app.html>.
2. Inspect the compact icon buttons in the instructions/test-data/import-export areas.
3. Expand **Test Data**.
4. Generate rows from a schema to enable stored-schema actions.
5. Inspect enabled `button[data-role]` controls for visible text, `aria-label`, and `title`.

Expected

Every icon-only actionable button has a programmatic accessible name, for example via `aria-label` or `title`, matching the user action.

Actual

The following visible enabled buttons were observed with empty visible text, no `aria-label`, and no `title`:

```
[
  { "role": "instructions-action-button", "disabled": false },
  { "role": "stored-schemas-save-as", "disabled": false },
  { "role": "stored-schemas-recover-draft", "disabled": false },
  { "role": "stored-schemas-load-last-used", "disabled": false },
  { "role": "stored-schemas-clear-last-used", "disabled": false },
  { "role": "download-button", "disabled": false }
]
```

Additional compact buttons such as embedded schema load/save file buttons were also observed as icon-only without names during the initial app-page control scout.

Functional Control Check

The embedded test-data workflow itself worked after expanding **Test Data**:

1. Filled schema **Status**, enum, **Open**, **Closed**.
2. Generated three rows.
3. Confirmed **Total rows**: 3 and grid header **Status**.
4. Used grid-to-text and text-to-grid roundtrip successfully.

This defect is therefore about accessible naming and discoverability rather than broken generation.

Evidence

- `../screenshots/app-initial.png`
- `../screenshots/app-generate-new-table.png`
- `../screenshots/app-grid-text-roundtrip.png`

Impact

Recent accessibility fixes may not cover the compact app-embedded controls. Screen reader users can encounter unnamed buttons, and sighted keyboard users get little contextual feedback beyond icons.

Collated Defect File: issue-228-bad-enum-quote-generates-corrupted-values.md

Bad Enum Quote Generates Corrupted Values Instead Of Failing

Summary

Malformed enum quote syntax is accepted in both generator Preview and app Generate. Instead of failing validation, the deployed app generates corrupted values such as `\\\\"active` and may report generation success.

Environment

- URL: <https://eviltester.github.io/grid-table-editor/>
- Deployment branch: `codex/228-improve-command-definition`
- Deployment commit: `a3b39ddcfe0f`
- Deployment build: `2026-06-24T23:03:43.621Z`

Steps To Reproduce

Generator:

1. Open <https://eviltester.github.io/grid-table-editor/generator.html>.
2. Click **Edit** as **Text**.
3. Enter:

```
status
datatype.enum(values="active,pending)
```
4. Click **Preview**.

App:

1. Open <https://eviltester.github.io/grid-table-editor/app.html>.
2. Expand **Test Data**.
3. Use the text schema editor with the same schema.
4. Generate data.

Expected Result

The malformed quote should be rejected before generation with a visible row-level error that identifies the unclosed quote or malformed `values` argument.

Actual Result

- Generator Preview generated rows including corrupted value `\\\\"active`.

- App Generate reported `Generate complete. Grid updated. and populated values including \\"active.`

Evidence

Supporting raw evidence is in:

- `../negative-validation-matrix-results.json`
- `../negative-validation-recheck-results.json`
- `../negative-validation-app-sample-results.json`
- `negative-validation-defects.md`

Repeatability

Repeated by the negative-validation subagent from a clean generator tab and in the app flow.

Collated Defect File: issue-228-boolean-probability-out-of-range-is-accepted.md

Boolean Probability Out Of Range Is Accepted

Summary

`datatype.boolean(probability=2)` and `datatype.boolean(probability=-0.1)` generate data despite the documented probability range of 0 to 1.

Environment

- URL: <https://eviltester.github.io/grid-table-editor/>
- Deployment branch: `codex/228-improve-command-definition`
- Deployment commit: `a3b39ddcfe0f`
- Deployment build: `2026-06-24T23:03:43.621Z`

Steps To Reproduce

1. Open `generator.html`.
2. Click `Edit as Text`.
3. Enter either schema:

```
flag
datatype.boolean(probability=2)
```

```
flag
datatype.boolean(probability=-0.1)
```

4. Click Preview.
5. Repeat probability=2 in app.html Test Data Generate.

Expected Result

Values outside the documented 0 to 1 probability range should be rejected before generation with a visible validation message.

Actual Result

- probability=2 generated all true.
- probability=-0.1 generated all false.
- App Generate reported success for probability=2.

Evidence

Supporting raw evidence is in:

- ../negative-validation-matrix-results.json
- ../negative-validation-recheck-results.json
- ../negative-validation-app-sample-results.json
- negative-validation-defects.md

Repeatability

Repeated by the negative-validation subagent in generator Preview and sampled in the app flow.

Collated Defect File: issue-228-controls-missing-accessible-names.md

Generator and app controls expose empty accessible names

Summary

Chrome's accessibility tree exposes empty names for several important deployed controls in the generator and app pages. The confirmed empty-name controls include the generator schema field type combobox, generator table preview searchbox, and app grid searchbox.

Environment

- URLs:
 - <https://eviltester.github.io/grid-table-editor/generator.html>
 - <https://eviltester.github.io/grid-table-editor/app.html>
- Date/time: 2026-06-25 00:13 Europe/London
- Viewport: mobile portrait, 360 x 740, device scale factor 2
- Browser: Chrome via DevTools Protocol with a temporary profile
- Related scope: issue #228 / PR #243 responsive and accessibility retest

Steps To Reproduce

1. Open <https://eviltester.github.io/grid-table-editor/generator.html>.
2. Inspect the Chrome accessibility tree for focusable controls.
3. Repeat on <https://eviltester.github.io/grid-table-editor/app.html>.
4. Compare control roles and accessible names.

Expected Result

Focusable controls should expose meaningful accessible names that describe their purpose. Examples:

- The generator schema type combobox should communicate that it selects the field type or method.
- The generator preview grid searchbox should communicate that it filters/searches the preview table.
- The app grid searchbox should communicate that it filters/searches the table column/grid.

Actual Result

Chrome accessibility tree returned these empty-name focusable controls:

`generator.html`

- `role=combobox`, empty name, `focusable=true`, `hasPopup=menu`
- `role=searchbox`, empty name, `focusable=true`, `editable=plaintext`

`app.html`

- `role=searchbox`, empty name, `focusable=true`, `editable=plaintext`

The route sweep also found adjacent unlabeled DOM controls in the same areas, including schema row inputs that rely on placeholders and table/search controls with no programmatic name.

Evidence

- Structured run data: `../responsive-accessibility-cdp-results.json`
- Generator mobile screenshot: `../screenshots/responsive-a11y-grid-table-editor-generator-html`

- App mobile screenshot: `../screenshots/responsive-a11y-grid-table-editor-app-html-mobile-po`

Severity

Medium. These are focusable controls in core app/generator workflows, and empty accessible names make them difficult or impossible to understand with assistive technology.

Notes

This finding is based on Chrome's accessibility tree, not only DOM heuristics.

Collated Defect File: `issue-228-empty-or-missing-enum-values-have-weak-feedback.md`

Empty Or Missing Enum Values Have Weak Feedback

Summary

Empty or missing enum values produce inconsistent and weak feedback across generator and app flows.

Environment

- URL: `https://eviltester.github.io/grid-table-editor/`
- Deployment branch: `codex/228-improve-command-definition`
- Deployment commit: `a3b39ddcfe0f`
- Deployment build: `2026-06-24T23:03:43.621Z`

Steps To Reproduce

Generator:

1. Open `generator.html`.
2. Click `Edit as Text`.
3. Enter either:

```
status
datatype.enum(values="")

status
datatype.enum()
```

4. Click **Preview**.

App:

1. Open `app.html`.
2. Expand **Test Data**.
3. Generate with `datatype.enum(values="")`.

Expected Result

Both generator and app should show a visible, actionable validation message, such as `Row 1: datatype.enum requires at least one non-empty value`.

Actual Result

- Generator Preview produced no output and no visible validation message.
- App Generate reported only **Generate failed. Check console for details.** and retained previous grid data.

Evidence

Supporting raw evidence is in:

- `../negative-validation-matrix-results.json`
- `../negative-validation-recheck-results.json`
- `../negative-validation-app-sample-results.json`
- `negative-validation-defects.md`

Repeatability

Repeated by the negative-validation subagent in generator Preview and sampled in the app flow.

Collated Defect File: `issue-228-generator-schema-row-tab-order-trap.md`

Generator schema row tab order traps focus before method/value controls

Summary

On the deployed PR #243 build, keyboard users cannot reliably tab through the generator schema row. From the **Column Name** input, Tab moves focus to body, then back to the row action buttons and the same **Column Name** input,

repeatedly. Focus does not advance to the field type picker, the value/regex input, schema constraints, or generation controls.

Environment

- URL: <https://eviltester.github.io/grid-table-editor/generator.html>
- Date/time: 2026-06-25 00:13 Europe/London
- Viewport: mobile portrait, 360 x 740, device scale factor 2
- Browser: Chrome via DevTools Protocol with a temporary profile
- Related scope: issue #228 / PR #243 responsive and accessibility retest

Steps To Reproduce

1. Open <https://eviltester.github.io/grid-table-editor/generator.html>.
2. Use a 360 x 740 mobile viewport.
3. Focus the schema row **Column Name** text input.
4. Press **Tab** repeatedly.

Expected Result

Focus should move forward through the schema row controls in page order, including the field type picker, the value/regex input, schema constraints, Add Field, stored schema controls, and then the generation controls.

Actual Result

Focus cycles back through the row action buttons and the same column-name field. The confirmed trace from a clean page was:

```
start: input text "Column Name"
Tab 1: body
Tab 2: button "Drag field to reorder"
Tab 3: button "Insert field after this row"
Tab 4: button "Remove field"
Tab 5: input text "Column Name"
Tab 6: body
Tab 7: button "Drag field to reorder"
Tab 8: button "Insert field after this row"
```

Evidence

- Structured run data: [../responsive-accessibility-cdp-results.json](#)
- Screenshot after the targeted tab-order recheck: [../screenshots/responsive-a11y-generator-tab-aft](#)
- Baseline mobile generator screenshot: [../screenshots/responsive-a11y-grid-table-editor-generat](#)

Severity

Medium. This blocks keyboard-only access to important generator schema controls and is directly tied to the PR's accessibility and picker-related risk areas.

Notes

The same repeating pattern appeared in the broader desktop and mobile tab traces captured during the route sweep.

Collated Defect File: issue-228-invalid-number-and-date-constraints-generate-error-rows.md

Invalid Number And Date Constraints Generate Error Rows

Summary

Invalid numeric/date constraints generate literal ****ERROR**** rows instead of failing validation before generation.

Environment

- URL: <https://eviltester.github.io/grid-table-editor/>
- Deployment branch: `codex/228-improve-command-definition`
- Deployment commit: `a3b39ddcfe0f`
- Deployment build: `2026-06-24T23:03:43.621Z`

Steps To Reproduce

Generator:

1. Open `generator.html`.
2. Click `Edit as Text`.
3. Enter:

```
n
number.int(min=1,max=10,multipleOf=0)
```
4. Click `Preview`.
5. Repeat with:

```
d
date.recent(days=-7)
```

App:

1. Open `app.html`.
2. Expand `Test Data`.
3. Generate with `number.int(min=1,max=10,multipleOf=0)`.

Expected Result

Invalid constraints should fail validation before generation. The app should not report success when output contains internal ****ERROR**** placeholders.

Actual Result

- Generator Preview produced ****ERROR**** rows for both invalid rules.
- App Generate reported `Generate complete. Grid updated.` for `multipleOf=0` and inserted ****ERROR**** rows.

Evidence

Supporting raw evidence is in:

- `../negative-validation-matrix-results.json`
- `../negative-validation-recheck-results.json`
- `../negative-validation-app-sample-results.json`
- `negative-validation-defects.md`

Repeatability

Repeated by the negative-validation subagent in generator Preview and sampled in the app flow.

Collated Defect File: issue-228-modal-focus-not-restored.md

Modal focus is not restored to trigger after Escape close

Severity

Medium

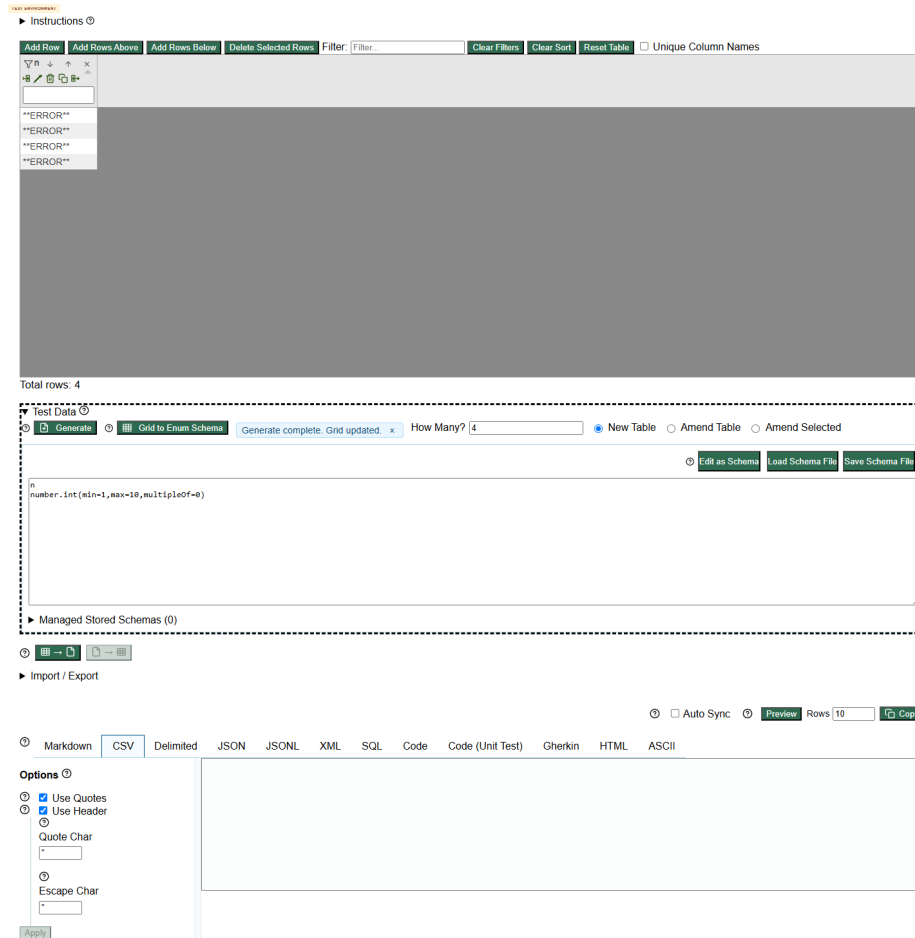


Figure 1: App multipleOf zero error success

Area

`generator.html` method picker and params editor modal keyboard/focus paths.

Environment

Deployed target: <https://eviltester.github.io/grid-table-editor/generator.html>

Date observed: 2026-06-25

Summary

Closing the method picker or params editor with Escape closes the modal but leaves focus on `<body>` instead of returning focus to the control that opened the modal. Keyboard and assistive-technology users lose their place in the schema row after dismissing either modal.

Reproduction - Method Picker

1. Open <https://eviltester.github.io/grid-table-editor/generator.html>.
2. Change the first schema row type to `faker`.
3. Focus `Select faker` command.
4. Press Enter to open the method picker.
5. Observe focus moves to the picker search input.
6. Press Escape.

Expected

The picker closes and focus returns to the `Select faker` command button that opened it.

Actual

The picker closes and focus lands on `<body>`.

Observed focus state after Escape:

```
{
  "modalVisible": false,
  "active": {
    "tag": "BODY",
    "cls": "theme-light"
  }
}
```

Reproduction - Params Editor

1. Open <https://eviltester.github.io/grid-table-editor/generator.html>.

2. Change the first schema row type to `faker`.
3. Open the method picker.
4. Switch to `All`, search `number.int`, select it, and Apply.
5. Focus the params button.
6. Press Enter to open the params editor.
7. Observe focus moves to `min value`.
8. Press Escape.

Expected

The params editor closes and focus returns to the params button that opened it.

Actual

The params editor closes and focus lands on `<body>`.

Observed focus state after Escape:

```
{
  "modalVisible": false,
  "active": {
    "tag": "BODY",
    "cls": "theme-light"
  }
}
```

Evidence

- `../screenshots/method-picker-keyboard-after-esc.png`
- `../screenshots/params-editor-keyboard-after-esc.png`

Impact

The modal workflows are keyboard-operable enough to open and close, but users must manually rediscover their location in the form after closing. This is especially frustrating in multi-row schemas and creates an accessibility regression risk.

Collated Defect File: issue-228-unknown-enum-named-argument-is-accepted.md

Unknown Enum Named Argument Is Accepted

Summary

`datatype.enum` accepts an unknown named argument `valuez` instead of rejecting it. Generation proceeds without a visible validation error.

Environment

- URL: <https://eviltester.github.io/grid-table-editor/generator.html>
- Deployment branch: `codex/228-improve-command-definition`
- Deployment commit: `a3b39ddcfe0f`
- Deployment build: `2026-06-24T23:03:43.621Z`

Steps To Reproduce

1. Open `generator.html`.
2. Click `Edit as Text`.
3. Enter:

```
status
datatype.enum(valuez="active,pending")
```

4. Click `Preview`.

Expected Result

The command should reject `valuez` as an unknown argument and show an actionable message such as `Unknown argument "valuez"; expected "values"`.

Actual Result

Generation proceeds with no visible validation error. The subagent observed output in both the initial matrix and clean recheck.

Evidence

Supporting raw evidence is in:

- `../negative-validation-matrix-results.json`
- `../negative-validation-recheck-results.json`
- `negative-validation-defects.md`

Repeatability

Repeated by the negative-validation subagent in generator Preview.

Collated Defect File: negative-validation-defects.md

Negative Validation Defects

Summary

Several malformed or invalid schema rules are accepted, silently fail, or generate ****ERROR**** data instead of producing useful validation messages. The affected behavior was observed on the deployed PR #243 test environment only.

- Environment: <https://eviltester.github.io/grid-table-editor/>
- Branch: `codex/228-improve-command-definition`
- Commit: `a3b39ddcfe0f`
- Build: 2026-06-24T23:03:43.621Z
- Evidence files:
 - `../negative-validation-matrix-results.json`
 - `../negative-validation-recheck-results.json`
 - `../negative-validation-app-sample-results.json`
 - `../screenshots/negative-validation-app-multipleof-zero-error-success.png`

Confirmed Behaviors

1. Bad enum quote generates corrupted values instead of failing

Schema:

```
status
datatype.enum(values="active,pending)
```

Generator Preview:

- Generated rows such as `\\\"active` and `pending`.
- No visible validation error appeared.

Main app Generate:

- Reported `Generate complete. Grid updated.`
- Updated the grid with values including `\\\"active`.

Expected:

- The malformed quote should be rejected before generation with a visible row-level error that identifies the unclosed quote or malformed `values` argument.

2. Unknown enum named argument is accepted

Schema:

```
status
datatype.enum(valuez="active,pending")
```

Generator Preview:

- Generated data with no visible validation error.
- Initial matrix produced values including `valuez=active` and `pending`; clean recheck produced `pending` values.

Expected:

- Unknown named argument `valuez` should be rejected for `datatype.enum`, with a message such as `Unknown argument "valuez"; expected "values"`.

3. Boolean probability accepts values outside documented range

Schemas:

```
flag
datatype.boolean(probability=2)

flag
datatype.boolean(probability=-0.1)
```

Generator Preview:

- `probability=2` generated all `true`.
- `probability=-0.1` generated all `false`.
- No visible validation error appeared.

Main app Generate:

- `probability=2` reported `Generate complete. Grid updated. and generated all true.`

Expected:

- Values outside the documented 0 to 1 probability range should be rejected before generation.

4. Invalid numeric/date constraints generate ****ERROR**** rows

Schemas:

```
n
number.int(min=1,max=10,multipleOf=0)
```

```
d
date.recent(days=-7)
```

Generator Preview:

- Both rules generated ****ERROR**** rows with no visible validation message.

Main app Generate:

- `number.int(min=1,max=10,multipleOf=0)` reported **Generate complete. Grid updated.** and inserted ****ERROR**** rows into the grid.

Expected:

- Invalid constraints should fail validation before generation.
- The app should not report success when generated rows contain internal ****ERROR**** placeholders.

5. Empty or missing enum values have inconsistent/weak feedback

Schemas:

```
status
datatype.enum(values="")
```

```
status
datatype.enum()
```

Generator Preview:

- Produced no output.
- No visible validation error appeared.

Main app Generate for `values=""`:

- Reported **Generate failed. Check console for details.**
- Left the previous grid data visible.

Expected:

- Both generator and app should show a visible, actionable validation message, e.g. **Row 1: datatype.enum requires at least one non-empty value.**

Severity

Medium.

Rationale: core generation flows remain available, but malformed schemas can produce corrupted data, successful-looking ****ERROR**** output, or non-actionable failures across the generator/app flows touched by PR #243 validation work.

Notes

Some invalid inputs did produce useful validation messages and are not filed as defects here:

- `number.int(min=10,max=1)` correctly reported that `min` must be less than or equal to `max`.
- `helpers.arrayElement(values="active,pending")` correctly failed as unsafe/complex faker syntax.
- Stray enum commas failed visibly, though the wording could be improved.